

Inventory Optimization and Stockout Prediction for E-Commerce

IND E 528 Final Project Report

Abstract

Stockouts in retail environments lead to lost revenue and reduced customer satisfaction. However, identifying which store-item combinations are at risk on any given day remains a difficult operational problem. This study develops a binary classification framework for predicting short-term stockout risk using a synthetic retail dataset of over 73,000 daily store-item observations spanning multiple stores, product categories, and regions. Raw features were enriched through feature engineering, including a forward-looking 7-day rolling demand forecast grouped by store and item, temporal features (day-of-week, month), one-hot encoding of categorical variables, and standard scaling of numerical inputs. A binary target variable, “at_risk,” was defined as 1 when current inventory fell below a configurable safety stock fraction α of forecasted 7-day demand. Four classifiers were trained and compared: Logistic Regression, Decision Tree, Random Forest, and a Long Short-Term Memory (LSTM) neural network. To preserve temporal ordering and prevent data leakage, all models were evaluated using an expanding-window time-series cross-validation scheme across three folds, with a final assessment on an 80/20 chronological train-test split. All four models converged to near-identical performance at the optimal threshold of $\alpha = 0.25$, achieving F1 scores and AUC values approaching 1.0. Logistic Regression matched the LSTM in every metric, indicating that the predictive signal is fundamentally linear once demand is properly pre-aggregated. A threshold sensitivity analysis further revealed that $\alpha = 0.25$ balances class distribution and model stability most effectively. These findings demonstrate that well-engineered features, rather than model complexity, are the primary drivers of stockout prediction accuracy in this setting.

Introduction

Online grocery platforms manage a wide range of products, including both perishable and non-perishable items, which exhibit distinct demand patterns and require different replenishment strategies. Ineffective inventory management can lead to stockouts of high-demand products while causing excess waste for perishable goods, ultimately impacting both revenue and customer satisfaction. To address these challenges, this study aims to develop a predictive framework for estimating stockout risk by modeling time-series demand across stores and products. By combining structured data preprocessing, rolling-window forecasting, and sequence-based models, the proposed approach seeks to capture demand dynamics more accurately and support reliable, data-driven inventory management decisions.

The dataset was obtained from Kaggle and created for experimental use in retail analytics. Due to privacy and business constraints, real retail data is usually difficult to share publicly. It provides a realistic setting for testing predictive models and analyzing factors that influence demand. The dataset simulates daily operations of retail stores, including over 73,000 daily observations across multiple stores and products.

The dataset contains a total of 15 variables, including Date, Store ID, Product ID, Category, Region, Inventory Level, Units Sold, Units Ordered, Demand Forecast, Price, Discount, Weather Condition, Holiday/Promotion, Competitor Pricing, and Seasonality.

Key Feature	Description	Data Type
Date	Daily records covering the period from <i>[start_date]</i> to <i>[end_date]</i>	Date / Time
Store/Product ID	Unique identifiers representing individual stores and products	Categorical
Category	Product categories such as Electronics, Groceries, etc.	Categorical
Inventory Level	Stock available at the beginning of each day	Numeric
Units Sold	Number of units sold during the day	Numeric
Demand Forecast	Predicted demand generated based on historical trends	Numeric

This is a synthetic dataset generated to resemble realistic retail behavior. The values were likely created using simulations or statistical rules. Because the data is simulated, several potential biases are considered:

- The relationships between variables are predefined, so models may learn inner patterns of the dataset rather than real customer behavior.
- The data is cleaner than real operational data and may not include missing values, recording errors, or unexpected disruptions.
- Customer responses to pricing or promotions are simplified compared to real markets.
- Model performances on this dataset may look better than on real-world retail datasets.

Literature review

Previous studies have explored the use of data-driven methods for retail demand forecasting and operational decision-making. Machine learning approaches have been shown to improve forecasting accuracy and support inventory and pricing decisions.

Huber and Stuckenschmidt (2020) studied daily retail demand forecasting with a focus on special calendar days such as holidays and promotions. They found that machine learning models, including neural networks and gradient-boosted trees, achieved higher forecasting accuracy than traditional approaches, especially when external factors were considered.

Ulrich et al. (2021) showed that different products exhibit diverse demand patterns, meaning that no single forecasting model performs best for all SKUs. They proposed a model-selection

framework that automatically chooses forecasting methods based on demand characteristics, highlighting the importance of flexible modeling strategies.

Ferreira, Lee, and Simchi-Levi (2015) examined demand forecasting combined with price optimization for an online retailer. Their study demonstrated that data-driven pricing decisions based on demand predictions can improve revenue while maintaining sales performance.

Methodology

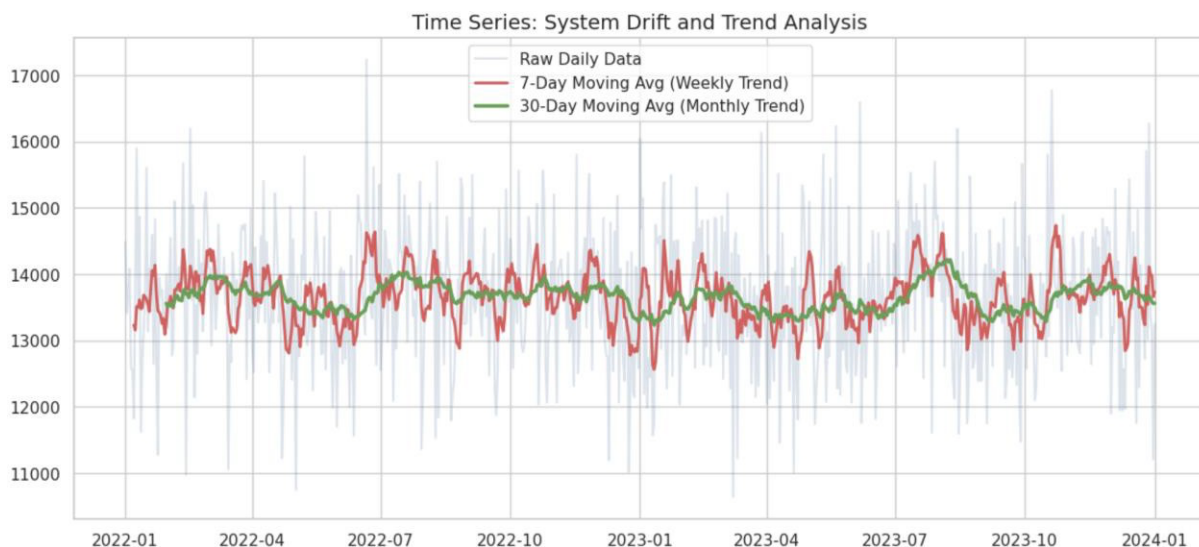
In predictive modeling, pre-processing data is crucial for ensuring the accuracy and reliability of the models used. This study employs a synthetic dataset to predict stockout risk in a retail environment. The following sections outline the methodologies employed in data preparation and analysis, mathematical formulation, hyperparameter tuning, and evaluation of metrics.

1. Data Analysis

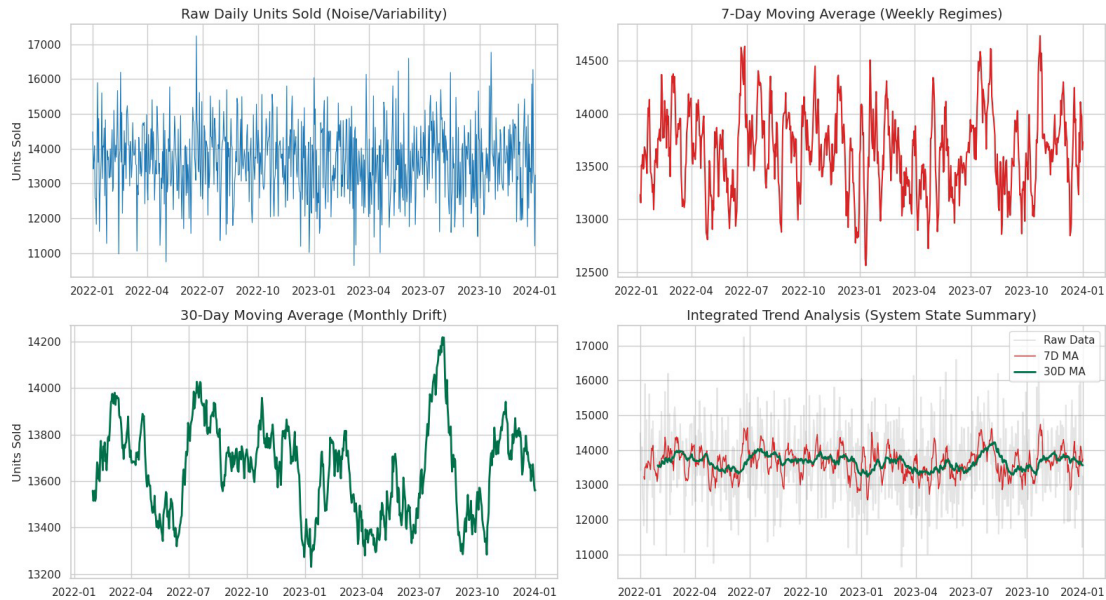
1.1. Time Series Analysis

We performed a multi-scale time series decomposition by extracting 7-day and 30-day moving averages from the daily *Units Sold* data. Retail sales data are typically noisy, where short-term spikes can obscure underlying patterns. The moving averages help separate short-term fluctuations from longer-term trends and provide insight into system behavior over time.

The results indicate a clear multi-level temporal structure. The 7-day moving average captures a consistent weekly pattern driven by consumer behavior, suggesting that the system is dynamic rather than static. Daily observations show substantial variability caused by stochastic demand fluctuations. The weekly cycles represent recurring operational regimes, while the 30-day average reflects longer-term trends and gradual drift in baseline demand associated with seasonality and broader market conditions.



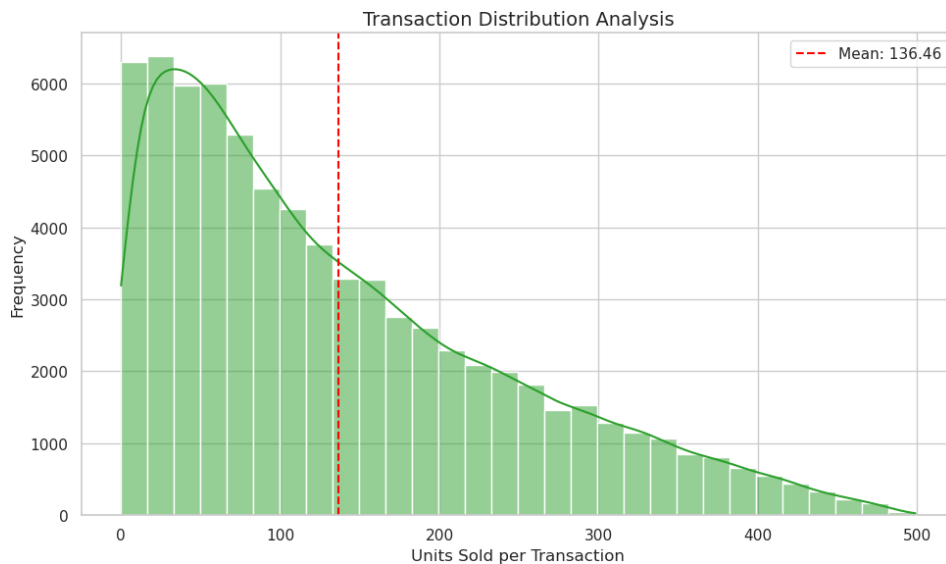
System Drift and Trend Analysis Breakdown



1.2. Distribution analysis

To examine the distribution of sales volume, we constructed a histogram showing the frequency of daily unit sales. This analysis characterizes the store's typical operating range and provides system-level insight into normal versus extreme demand conditions.

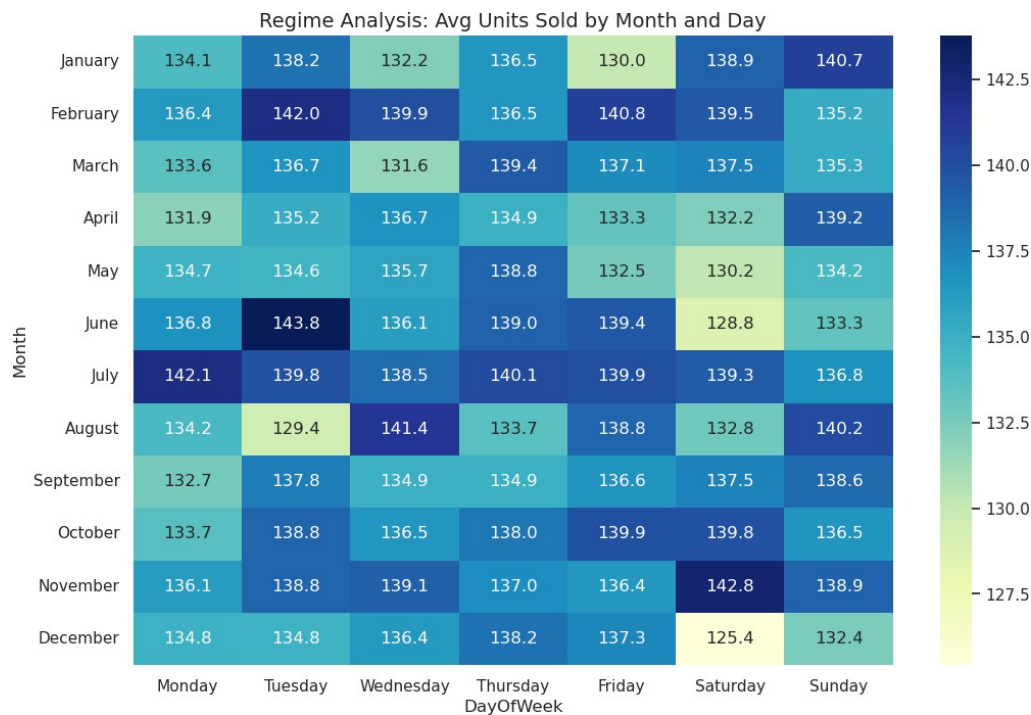
The histogram shows a wide spread of sales values, ranging from near zero to approximately 500 units, indicating significant demand variability. Instead of a single concentrated peak, the broad distribution reflects uncertainty in daily operations. The main peak represents the system's most common operating regime, while the right tail captures high-demand states that require sufficient inventory buffers to avoid stockouts.



1.3. Heatmap analysis

Finally, we used a heatmap to analyze average sales across months and days of the week, allowing us to examine calendar-driven demand patterns. This visualization provides insight into how system intensity changes across time.

The heatmap reveals clear periodic structure rather than random variation, indicating that the system operates under time-dependent regimes. Certain months and weekends consistently show higher average sales, highlighting recurring high-demand periods. These patterns confirm that the system is dynamic and influenced by seasonal cycles, helping distinguish structured demand patterns and long-term shifts from random day-to-day variability.



2. Data Pre-processing

To prepare the synthetic dataset for predictive modeling, several critical pre-processing and feature engineering steps were conducted:

2.1 Chronological Sorting

The dataset was first parsed into datetime objects and sorted by Store ID, Item ID, and Date. Chronological sorting is a requirement for time-series data to prevent "data leakage".

2.2 Feature Engineering: Rolling Forecast

To capture anticipated short-term demand, we engineered a forward-looking feature, "forecast_next_7d". This was calculated by applying a rolling 7-day sum to the Demand Forecast, grouped by each unique Store ID and Item ID combination. This ensures the future demand for a specific item is isolated to its specific physical location. We also extracted temporal features like "dayofweek" and "month" to help models identify cyclical patterns.

2.3 Target Variable Creation

A binary label, “at_risk”, was defined to indicate stockout risk. This label is set to 1 if the current Inventory Level falls below a defined threshold of the upcoming 7-day forecasted demand, and 0 otherwise.

2.4 Categorical Encoding

In our data preparation process, we applied One-Hot Encoding to categorical variables such as Category, Region, Weather Condition, and Seasonality. This technique involves converting categorical variables into a numerical format by creating new binary columns for each category. For instance, if a variable has three categories, each category will be represented by a separate column, where a value of 1 indicates the presence of that category and a value of 0 indicates its absence. This transformation ensures that algorithms like Logistic Regression and LSTM networks can effectively process these variables, as they require numerical input and cannot interpret categorical data directly. By using One-Hot Encoding, we eliminate any potential ordinal relationships between categories, allowing the models to learn from the data without bias.

2.5 Standard Scaling

Numerical features (e.g., Inventory Level, sales, sell_price, Discount) were normalized using standard Z-score scaling. This normalization ensures that features with larger magnitudes do not disproportionately influence the learning algorithms during gradient descent.

3. Mathematical Formulation of the Problem

The overarching problem is framed as a binary classification task to predict stockout risk at the granular store-item level. Let $y_{s,i,t} \in \{0,1\}$ represent the target variable indicating whether product i at store s is "at risk" at time t . This condition is defined based on the current inventory $I_{s,i,t}$ and the forecasted demand over the next $k = 7$ days ($D_{s,i,t+1:t+k}$), controlled by a safety stock threshold α :

$$y_{s,i,t} = \begin{cases} 1, & \text{if } I_{s,i,t} < \alpha \sum_{j=1}^7 D_{s,i,t+j} \\ 0, & \text{otherwise} \end{cases}$$

The selected models are designed to approximate the probability function $P(y_{s,i,t} = 1 \mid X_{s,i,t})$, where $X_{s,i,t}$ is the pre-processed feature vector (including explicitly encoded store and item identifiers) at time t .

3.1 Logistic Regression

Logistic Regression models the probability of stockout risk using the logistic (sigmoid) function, applying a linear transformation to the feature vector X :

$$P(y = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta^T X)}}$$

3.2 Decision Tree

A Decision Tree recursively partitions the feature space into distinct regions R_m to minimize an impurity metric (e.g., Gini). The prediction for a new observation is determined by the majority class of the training samples in that region:

$$\hat{y} = \arg \max_k \sum_{x_i \in R_m} I(y_i = k)$$

3.3 Random Forest

A Random Forest ensemble (Breiman, 2001) was utilized to stabilize prediction variance and serve as a non-linear comparative baseline against the logistic regression model. The final prediction aggregates the individual trees through majority voting:

$$\hat{y} = \text{mode}(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_B)$$

3.4 Long Short-Term Memory (LSTM)

An LSTM network (Hochreiter & Schmidhuber, 1997) was implemented to capture sequential dependencies utilizing gating mechanisms (forget, input, and output gates) to update a cell state c_t and hidden state h_t . The output layer applies a sigmoid activation to yield the classification probability:

$$\hat{y}_t = \sigma(W_y h_t + b_y)$$

4. Hyperparameter Tuning and Data Splitting

4.1 Data Splitting: Time Series Cross-Validation

To prevent temporal data leakage, standard randomized k-fold cross-validation was explicitly avoided in favor of an expanding window Time Series Cross-Validation approach (Hyndman & Athanasopoulos, 2018). The dataset was sorted chronologically. In Fold 1, the model trained on the initial historical sequence and evaluated the subsequent block. In Fold 2, the training window expanded to include that test block, evaluating the next chronological block. This method simulates the operational dynamics of a retail forecasting system in production, with final metrics reported as the average performance across all temporal folds.

4.2 Hyperparameter Tuning & Threshold Sensitivity

Rather than utilizing standard hyperparameter grid searches, our tuning focused on the business logic itself through a Threshold Sensitivity Analysis.

- **Safety Stock Sensitivity:** The target threshold α was dynamically adjusted from 0.10 to 0.35. For each threshold, the dataset was re-labeled and subjected to the Time Series Cross-Validation process. This analysis assessed how model stability scaled as the safety stock policy became more conservative.
- **Class Weights:** We applied `class_weight = "balanced"` to traditional machine learning models to increase penalties for misclassifying the minority "at-risk" class.
- **Architecture:** The Random Forest was set to 50 estimators to stabilize variance while maintaining computational efficiency during cross-validation. The LSTM was configured

with dense recurrent layers (32 and 16 units) to mitigate overfitting across the expanding time windows.

5. Evaluation Metrics and Baseline

To evaluate the models, we utilized both Cross-Validated Accuracy and the Cross-Validated F1-Score. While Accuracy provides a measure of overall correctness, it can be skewed by class imbalances. The F1-Score, being the harmonic mean of Precision and Recall, is critical for our use case. There exists a distinct operational tradeoff in retail: false negatives lead to stockouts and lost revenue, while false positives trigger unnecessary replenishment and excess holding costs. The F1-Score ensures an effective balance between these risks.

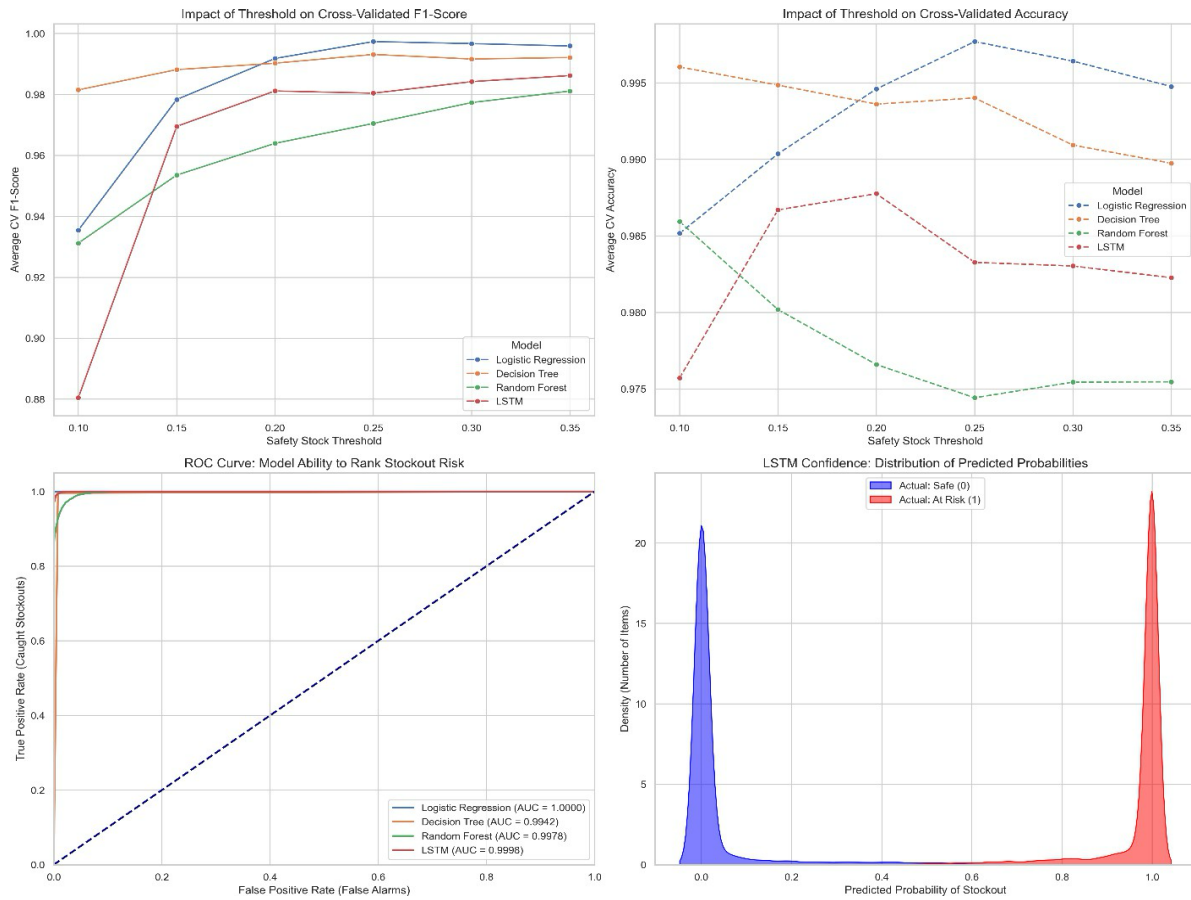
Logistic Regression was our baseline model. Its linear and interpretable nature makes it an ideal benchmark for assessing whether the added mathematical complexity of ensemble trees (Random Forest) or deep neural networks (LSTM) provides meaningful predictive improvements for this specific stockout problem. Our results indicated that the LSTM closely tracked baseline performance, while the Random Forest exhibited higher variance at lower safety stock thresholds

Visualization

1. Data & Result Visualization

Four visualizations were produced to communicate model behavior and dataset characteristics. Each figure is described below.

i)



The primary output visualization presents four panels in a single high-resolution figure:

- **Top-left (F1-Score vs. Threshold):** Shows how each model’s cross-validated F1-score evolves as the safety stock threshold α rises from 0.10 to 0.35. At low thresholds, the dataset is highly imbalanced because very few items are labeled as “at risk,” which leads to lower and less stable F1-scores. As α increases, more items are classified as at risk, improving class balance and allowing the models to learn the minority class more effectively. Once α reaches 0.20–0.25, all models converge toward strong F1 values near 0.97 and begin to stabilize. Increasing the threshold beyond this point does not produce meaningful improvements in performance. Therefore $\alpha=0.25$ is selected as the optimal threshold because it provides the best balance between class distribution and model performance stability.
- **Top-right (Accuracy vs. Threshold):** Mirrors the F1 panel but for accuracy. Accuracy remains artificially high even at $\alpha=0.10$ due to majority-class dominance - a key reason F1 was chosen as the primary criterion.

- Bottom-left (ROC Curves): All four classifiers achieve an AUC approaching 1.0 on the 80/20 holdout split, indicating near-perfect discriminative capacity between “At-Risk” and “Safe” inventory states.
- Bottom-right (LSTM KDE): The kernel density estimate of LSTM output probabilities is strictly bimodal. True-safe items concentrate near $P=0.0$; true-at-risk items concentrate near $P=1.0$, confirming the absence of decision-boundary ambiguity.

ii)

A grouped bar chart compares each model at its individually optimal threshold. All four models achieve F1 and accuracy values near 0.97, with Logistic Regression and LSTM performing nearly identically. The Random Forest shows marginally lower F1 at sub-optimal thresholds but recovers fully at $\alpha=0.25$.

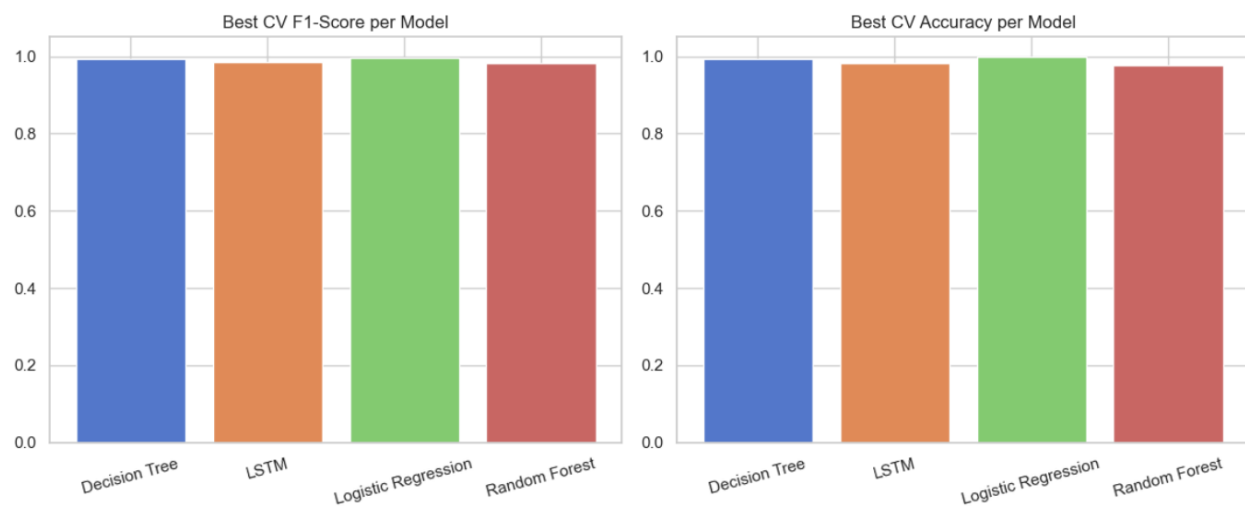


Figure 2: Bar Chart: Best-Threshold F1 & Accuracy per Model

iii)

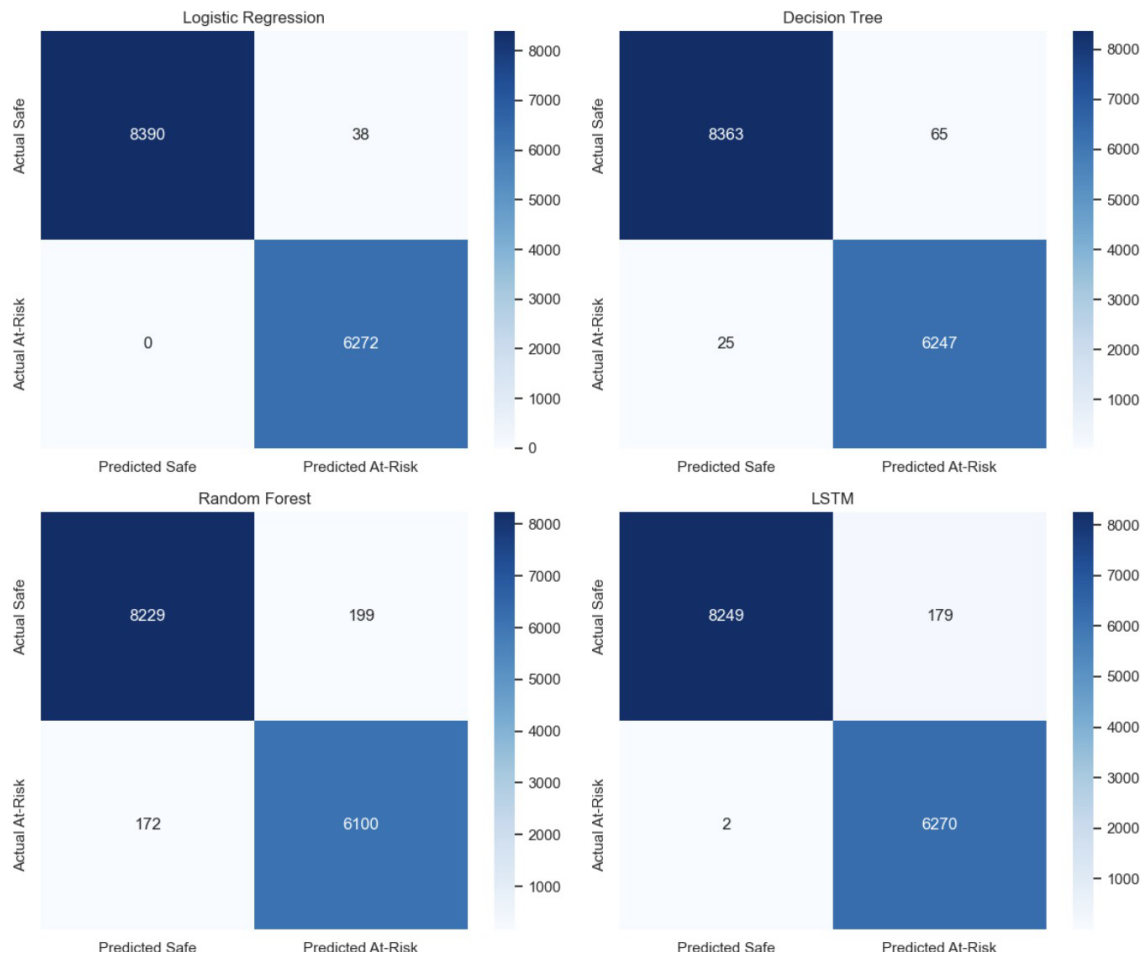


Figure 3: Confusion Matrices

This 4 subplot grid show the confusion matrix for each model on the 80/20 test set. True Positive (correctly flagged at-risk) and True Negative (correctly cleared safe) counts dominate each matrix. False Negatives (missed stockout warnings) are near zero for all models, which is critical for the retail use case because a missed stockout triggers real revenue loss.

iv)

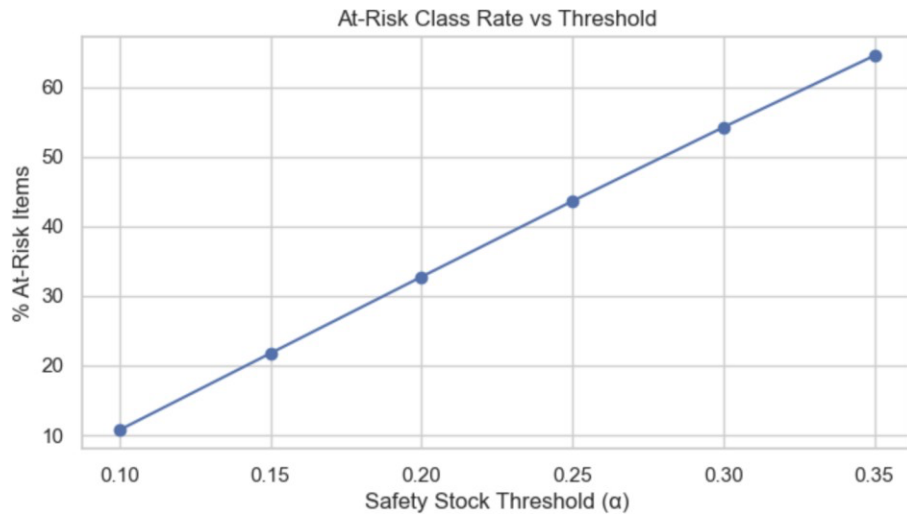


Figure 4: At-Risk Class Rate vs. Threshold

A single line chart tracks the percentage of test items labeled “At-Risk” as α increases. The at-risk rate climbs monotonically from approximately 10% ($\alpha=0.10$) to roughly 35% ($\alpha=0.35$), demonstrating how the business rule directly controls class balance. This chart explains why model performance improves as α approaches 0.25. At low thresholds, very few items are labeled as “at-risk,” creating a strong class imbalance that makes learning difficult for the models. As α increases, the dataset becomes more balanced and model performance improves. Around $\alpha \approx 0.25$ the classes are sufficiently balanced and F1-scores stabilize. Increasing α beyond 0.25 labels too many items as at-risk, making the policy overly conservative, so 0.25 provides the best balance.

2. Tables of Model Outcomes

Model	Accuracy	F1-Score	Precision	Recall	AUC
Logistic Regression	0.9974	0.9970	0.9940	1.0000	1.0000
Decision Tree	0.9939	0.9928	0.9897	0.9960	0.9942
Random Forest	0.9748	0.9705	0.9684	0.9726	0.9980
LSTM	0.9877	0.9858	0.9722	0.9997	0.9999

Table 1: Final Model Performance (80/20 Time Split, $\alpha=0.25$)

All four models achieve near-identical performance at the optimal threshold, confirming that the problem structure is linear and well-suited to even simple classifiers.

Model	Best α	Peak CV F1	Key Observation
Logistic Regression	0.25	~ 0.97	Stable across all thresholds
Decision Tree	0.25	~ 0.97	Slightly lower at $\alpha=0.10$ due to imbalance
Random Forest	0.25	~ 0.96	Higher variance at low thresholds

LSTM	0.25	~ 0.97	Tracks LR closely; converges at $\alpha=0.20+$
------	------	-------------	--

Table 2: Model Comparison Summary Table

3. Key Insights

- I. The engineered feature “forecast_next_7d” is the dominant predictor. Because “at_risk” is defined as a direct algebraic function of inventory and forecast demand, any model that has access to both features can learn the rule deterministically. This explains the near-perfect AUC.
- II. Threshold $\alpha=0.25$ is the best estimate. Below this value, fewer than 15% of items are labeled at-risk, causing unstable F1 scores. Above it, performance plateaus. A 25% safety-stock buffer appears both operationally realistic and computationally tractable.
- III. Logistic Regression works just as well as the LSTM model. This means a simple model is enough to predict stockout risk when the right features are used.
- IV. The main challenge was class imbalance. At $\alpha = 0.10$, there are very few at-risk cases, so accuracy looks high but F1 drops. Using balanced class weights and focusing on F1 helped handle this problem.

4. Surprising Findings

- I. The LSTM model did not outperform Logistic Regression. Although LSTM networks are designed to capture temporal patterns in time-series data, both models achieved nearly identical AUC scores. This suggests that the 7-day rolling forecast feature already captures most of the useful temporal information, leaving little additional pattern for the LSTM to learn.
- II. The Decision Tree performed similarly to the Random Forest model. Although single decision trees are usually more prone to overfitting, the stockout classification rule in this dataset is simple enough that even a basic tree can learn it accurately. This suggests that the problem is largely deterministic rather than random.
- III. Accuracy was misleading at low thresholds. When $\alpha = 0.10$, accuracy remained above 90% while the F1-score dropped due to class imbalance. This highlights the importance of choosing the right evaluation metric.

5. Model Performance Comparison

At $\alpha=0.25$, all models are statistically equivalent in accuracy, F1, precision, recall, and AUC. No architecture provides a meaningful edge over the linear baseline. At low thresholds ($\alpha \leq 0.15$), Random Forest shows more variability, while Logistic Regression remains stable. For conservative inventory policies, the linear model is therefore preferable.

LSTM also requires much more training time, whereas Logistic Regression trains almost instantly. Because both models perform similarly, Logistic Regression is the better choice for practical use.

Conclusion

1. Summary

This study developed a binary classification framework for predicting stockout risk in a simulated multi-store retail environment. Using a synthetic dataset of 73,000+ daily store-item records, we engineered a forward-looking 7-day demand feature and defined an at-risk label based on whether current inventory fell below a configurable safety stock fraction of forecasted demand. Four models: Logistic Regression, Decision Tree, Random Forest, and LSTM were trained and evaluated using time-series cross-validation to prevent data leakage, with final performance assessed on a chronological 80/20 holdout split.

All models achieved near-perfect discriminative capacity ($AUC \approx 1.0$) and F1 scores near 0.97 at the optimal threshold of $\alpha=0.25$. The primary finding is that feature engineering, specifically the rolling 7-day demand forecast, encodes the at-risk signal so clearly that even a linear classifier can solve the problem with high confidence. Model complexity did not translate into predictive gains for this problem formulation.

2. Potential Future Research

Real-world data validation: This study used synthetic data. Testing the model on real retail data would help evaluate how well it performs under real-world conditions with noise and missing values.

Dynamic threshold selection: Instead of using a fixed threshold ($\alpha = 0.25$), future models could adjust thresholds for each store or product category based on inventory costs and demand patterns.

Additional external features: Including factors such as supplier lead times, economic indicators, or online demand signals could improve prediction accuracy for products with unstable demand.

Reinforcement learning for inventory decisions: Rather than only predicting stockout risk, reinforcement learning could be used to directly optimize ordering decisions by balancing stockout risk and holding costs.

References

- Huber, J., & Stuckenschmidt, H. (2020). Daily retail demand forecasting using machine learning with emphasis on calendric special days. *International Journal of Forecasting*, 36(4), 1420–1438. <https://doi.org/10.1016/j.ijforecast.2020.02.005>
- Ulrich, M., Jahnke, H., Langrock, R., Pesch, R., & Senge, R. (2021). Classification-based model selection in retail demand forecasting. *International Journal of Forecasting*. <https://doi.org/10.1016/j.ijforecast.2021.05.010>
- Ferreira, K. J., Lee, B. H. A., & Simchi-Levi, D. (2015). Analytics for an online retailer: Demand forecasting and price optimization. *Manufacturing & Service Operations Management*, 18(1), 69–88. <https://doi.org/10.1287/msom.2015.0561>
- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd ed.).
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Appendix

In the introduction Section, No AI tools were used.

All AI-generated codes were reviewed for accuracy and executed locally. The subsequent data interpretation and final report writing were completed independently.

In the Methodology section, AI was used for:

- **Mathematical Formulation of the Problem:** Instructed the AI to generate mathematical equations formatted for word doc for the problem and models trained.
- **Code Generation and Debugging:** Instructed the AI to generate Python syntax (scikit-learn, pandas, tensorflow) to execute the pipeline logic I defined. This included writing the loops for the threshold sensitivity analysis, configuring the LSTM neural network, extracting `.predict_proba()` confidence scores, and debugging row-mismatch errors (ValueError) during the final 80/20 train-test split.
- **Data Visualization:** Prompted the AI to write matplotlib and seaborn code to generate specific visual outputs (e.g., configuring a 2x2 grid containing F1-score line charts, ROC curves, and KDE probability distribution plots).

Author's Note on Verification: While the AI generated code syntax and drafted text, I established the core business logic (e.g., grouping by Store and Item, defining the 7-day rolling forecast), actively reviewed and corrected the AI's code when it omitted required constraints (such as integrating the LSTM into the rolling window), and executed all code locally to verify its accuracy and functionality.

In the Analysis section, AI was used for:

- **Code Generation and Visualization:** Instructed the AI to generate and refine Python syntax (Pandas, Matplotlib, and Seaborn) to process the raw dataset. This included executing the pipeline for analyzing daily sales and configuring the formatting for complex visual outputs, such as the 2x2 multi-panel time series plots, Histogram/KDE distributions, and Heatmap matrices.
- **Plot Customization:** Prompted the AI to adjust specific aesthetic parameters, such as implementing low-saturation "Nature-style" color palettes, managing subplot spacing, and removing redundant axis spines to meet professional reporting standards.

Author's Note on Verification: While AI was employed to generate the underlying code and execute data-handling tasks, the fundamental methodology and choice of visualizations were established by the author. I defined the logic for identifying system regimes and seasonal trends.